

Exploring the command line

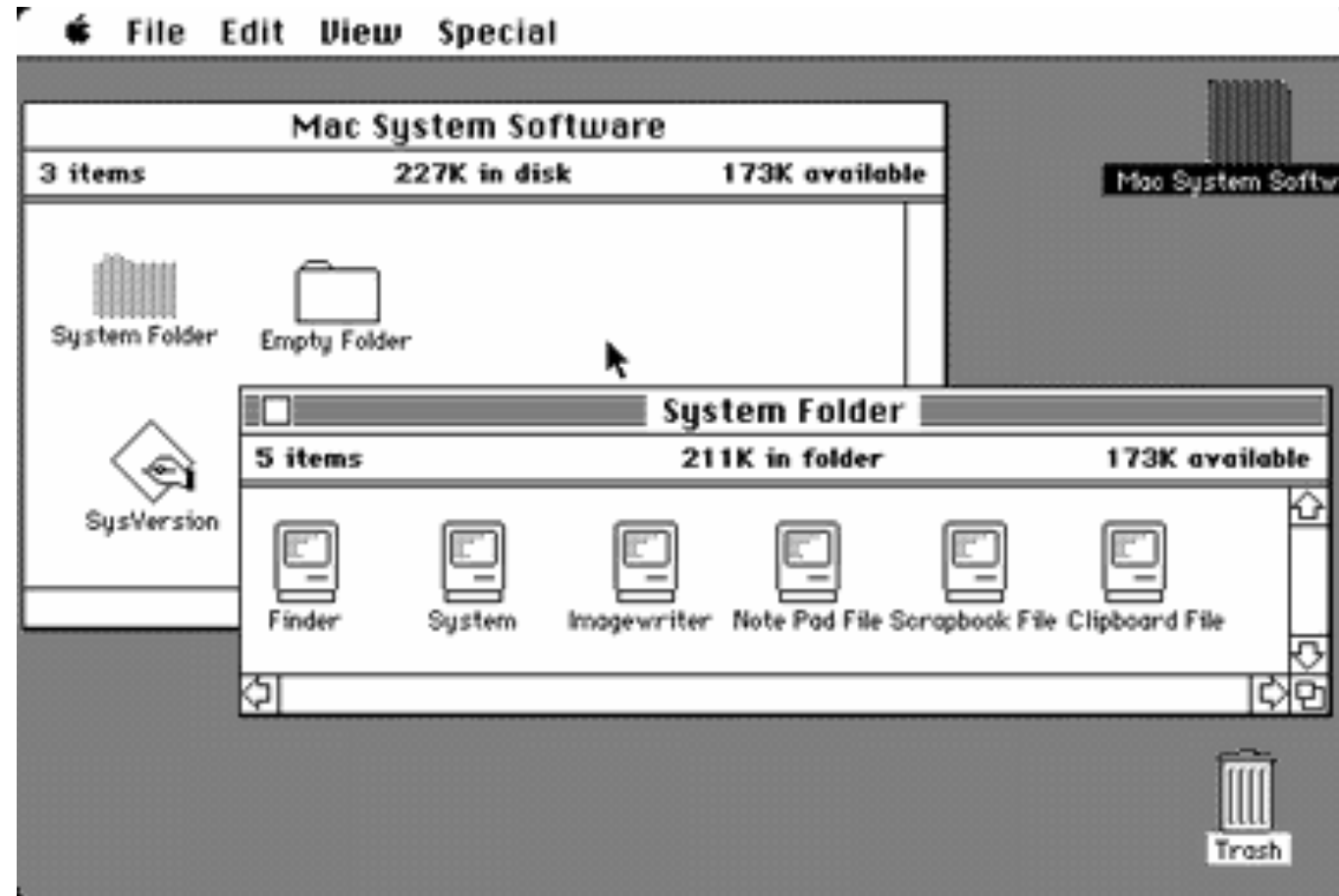
Massimiliano Carloni
(ACDH-ÖAW)

University of Vienna, 16 October 2025

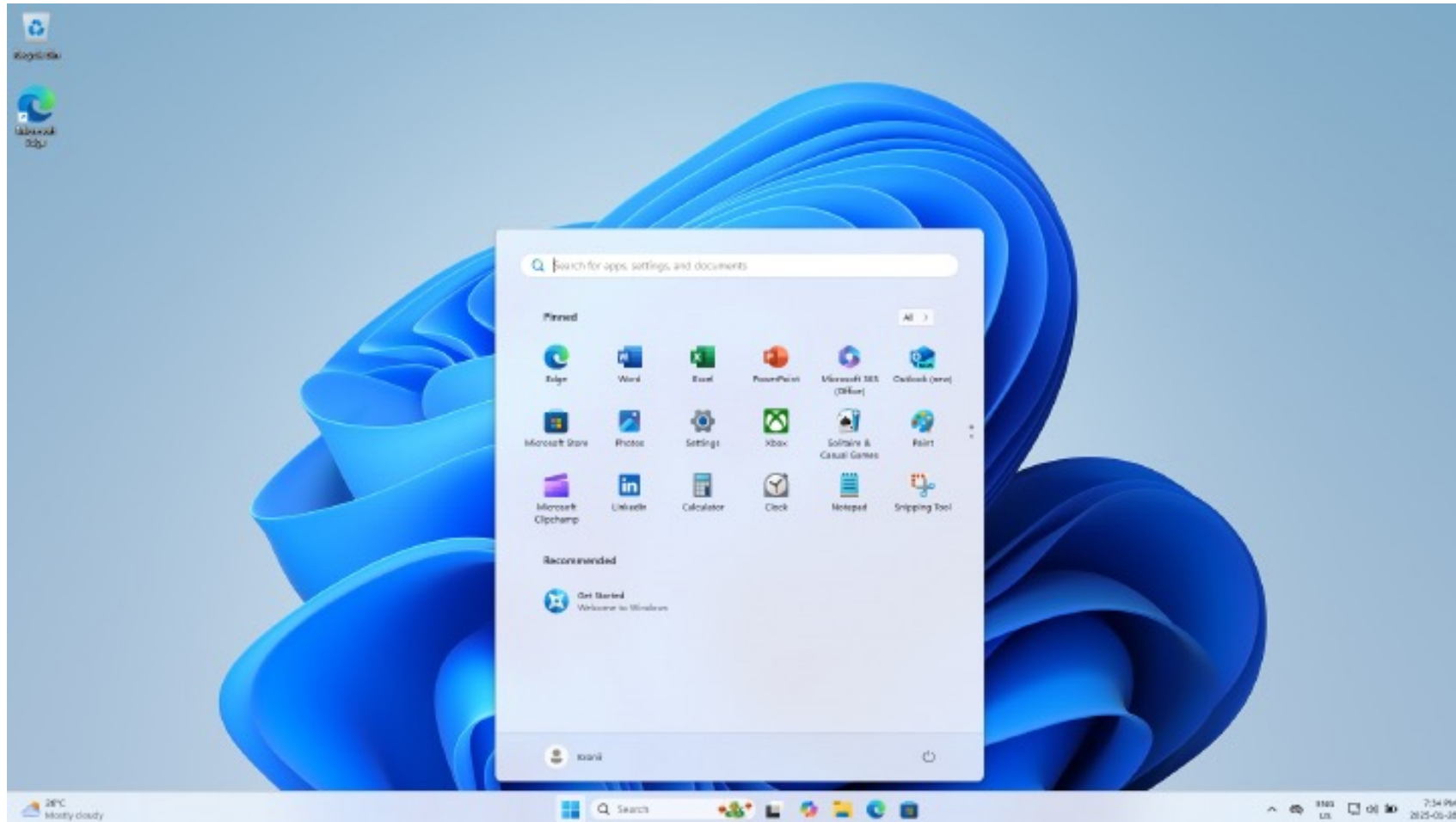


- since 2021 Research Data Engineer at the Austrian Centre for Digital Humanities
- Background in Classical Philology
- Graffiti project [INDIGO](#)
- Active in [DARIAH-EU](#) infrastructure
- EU infrastructure project [ATRIUM](#)

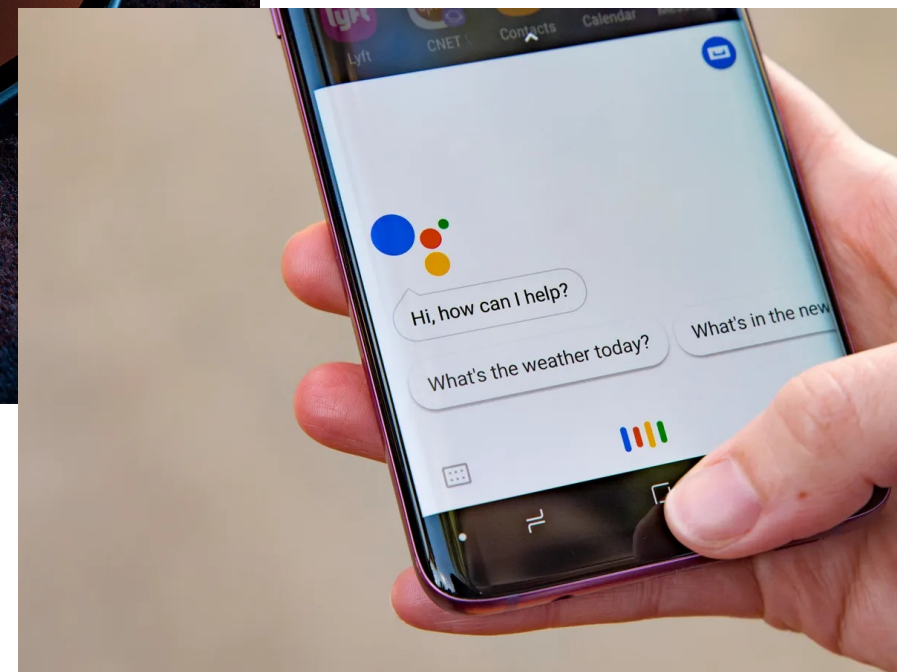
Graphical user interface



Graphical user interface



Voice interface



Brain-computer interface



Text-based interface (aka *command line* or *shell*)



Why the shell in 2025?

Installing software

Especially when
setting up a
container with
Docker

Example from
Editing Tools for
Digital Epigraphy



Installation

You may either

1. install the application into an existing eXist instance or
2. use the docker image

Using option 1, the application requires at least Java version 8 and eXist version 6.2.0. Details of the installation process are described in the [TEI Publisher documentation](#). You may skip the last step: *Installing TEI Publisher*, which is not needed for EDEp, but open the dashboard as described. Next, download the EDEp application and data package from the [GitHub release page](#). Install those one after the other: either click on the *Upload* button and select the downloaded `edep-*.xar` or drag and drop it onto the button.

Option 2 is **recommended** for new users and easier if you would like a working environment without having to install Java and eXist. You need docker installed though. Windows and Mac users may download the [docker desktop](#) application. After install, open a shell and run the following pull command:

```
docker pull ghcr.io/eeditiones/edep:latest
```

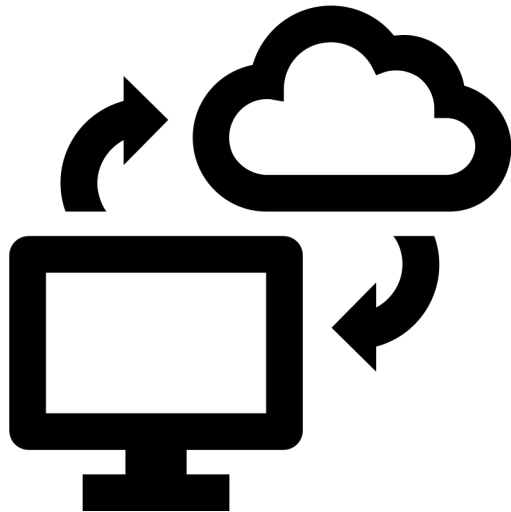


Afterwards, either start a container from the image using the run command:

```
docker run -p 8080:8080 --name edep ghcr.io/eeditiones/edep:latest
```



Connect to (university) server



Secure Shell Protocol (SSH Protocol)

```
ssh username@your_server_ip
```

Tinkering with tech stuff

SimonWaldherr,
CC BY-SA 4.0
<<https://creativecommons.org/licenses/by-sa/4.0/>>,
via Wikimedia
Commons




```
11:10:01 AM - nmap - host: 10.2.2.2 - [mobile]
11:10:01 AM - nmap - No exact OS matches for host.
11:10:01 AM - nmap - Hmap run
11:10:01 AM - nmap - B sshnake
11:10:01 AM - nmap - Connecting
11:10:01 AM - nmap - Attempting to exploit SSH01 CR032... successful.
11:10:01 AM - nmap - Resetting root password to "210H0101".
11:10:01 AM - nmap - System open: Access Level (9)
11:10:01 AM - nmap - B ssh 10.2.2.2 -1 root
11:10:01 AM - nmap - root@10.2.2.2's password:
11:10:01 AM - nmap - RRF-CONTROL> disable grid nodes 21-48; connect sector 11 (27 nodes)
11:10:01 AM - nmap - Warning: Disabling nodes 21-48 will disconnect sector 11 (27 nodes)
11:10:01 AM - nmap - ARE YOU SURE? (y/n) y
11:10:01 AM - nmap - Grid Node 21 offline...
11:10:01 AM - nmap - Grid Node 22 offline...
11:10:01 AM - nmap - Grid Node 23 offline...
```

Trinity uses nmap in *The Matrix Reloaded*
(YouTube: <https://youtu.be/0PxTAn4g20U>)

More tasks...

- Install Python packages

e.g. `pip install pandas`

- Setting up an environment variable

e.g. `export VARIABLE_NAME=content`

- Print a list of the file names in a folder

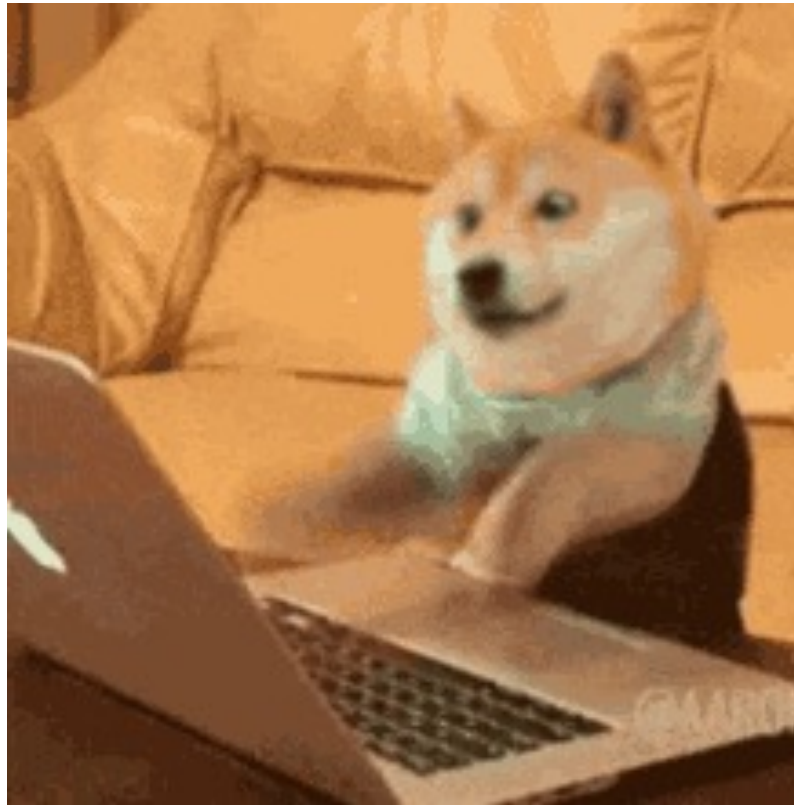
e.g. `ls folder_name > list.txt`

Some useful tools

- [exiftool](#) for reading and editing metadata in images
- **grep** (for using regular expressions → class on the 9th)
- **git** (we'll see it on the 29th)
- [pandoc](#) for converting between (a lot of) formats

For some of these tools, GUIs have been developed. But the command line tools are often more stable and more flexible

High action-to-keystroke ratio



Shell scripts



A good introduction to programming languages



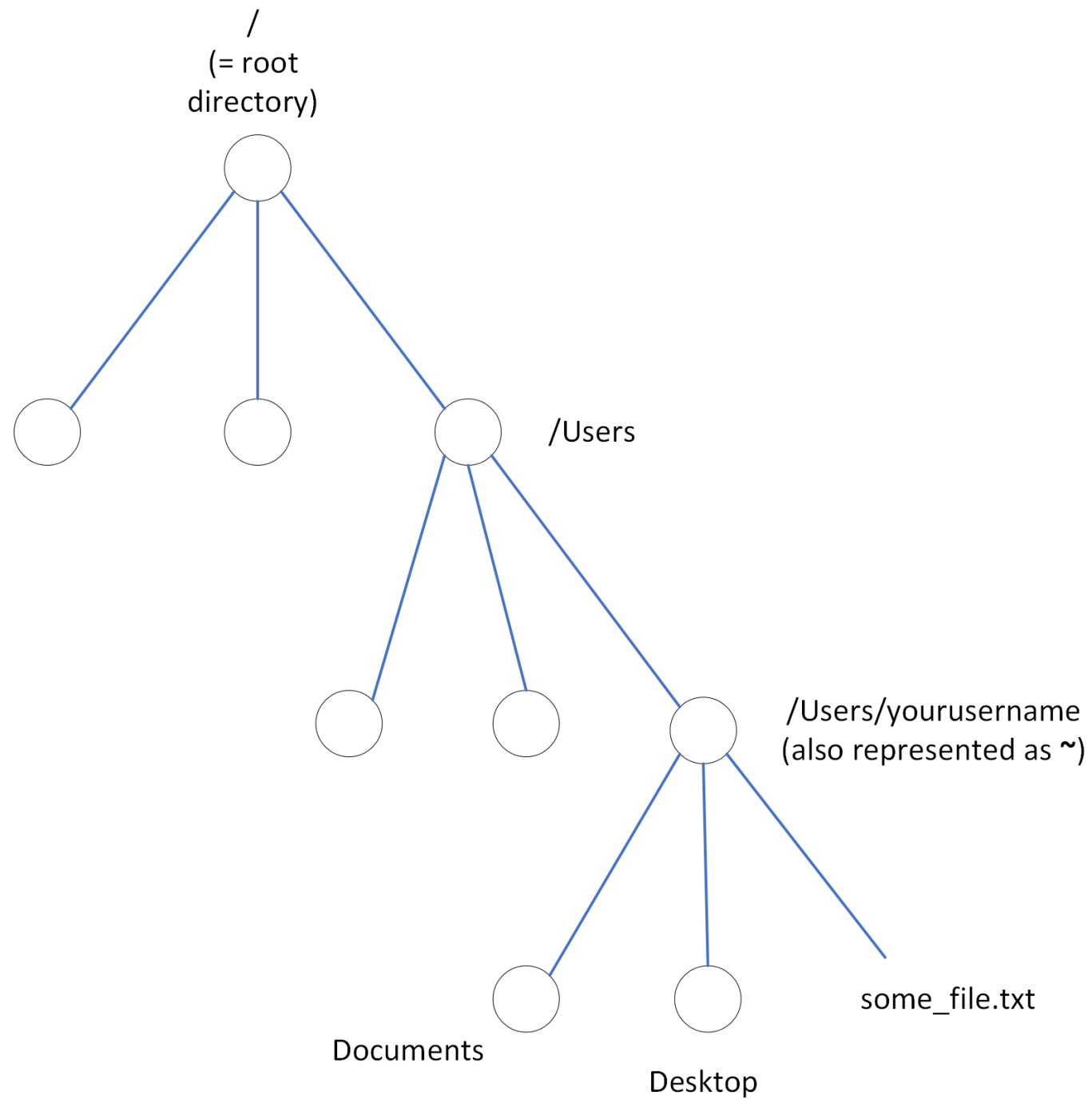
Let's try something out

First steps

- A first simple command: `whoami`
- Let's try with some gibberish: `asdfewqew`
 - The command line will tell you that it does not find the command
- The shell is case-sensitive
 - Sometimes, you might get the same result
 - Sometimes, you get different results (`Whoami` $\neq$ `whoami` on Mac)
- Check the current directory: `pwd`
- Where are we now?

File system

- You are probably going to be in your home directory
 - Directory ~ folder
 - In my case, /Users/mcarloni
- The home directory is where all your personal files are stored
- Inside your home directory, you will typically find several other directories
- Let's check it out: `ls`
 - Downloads, Movies, Music etc.
- Imagine this as an upside-down tree



Some warnings!

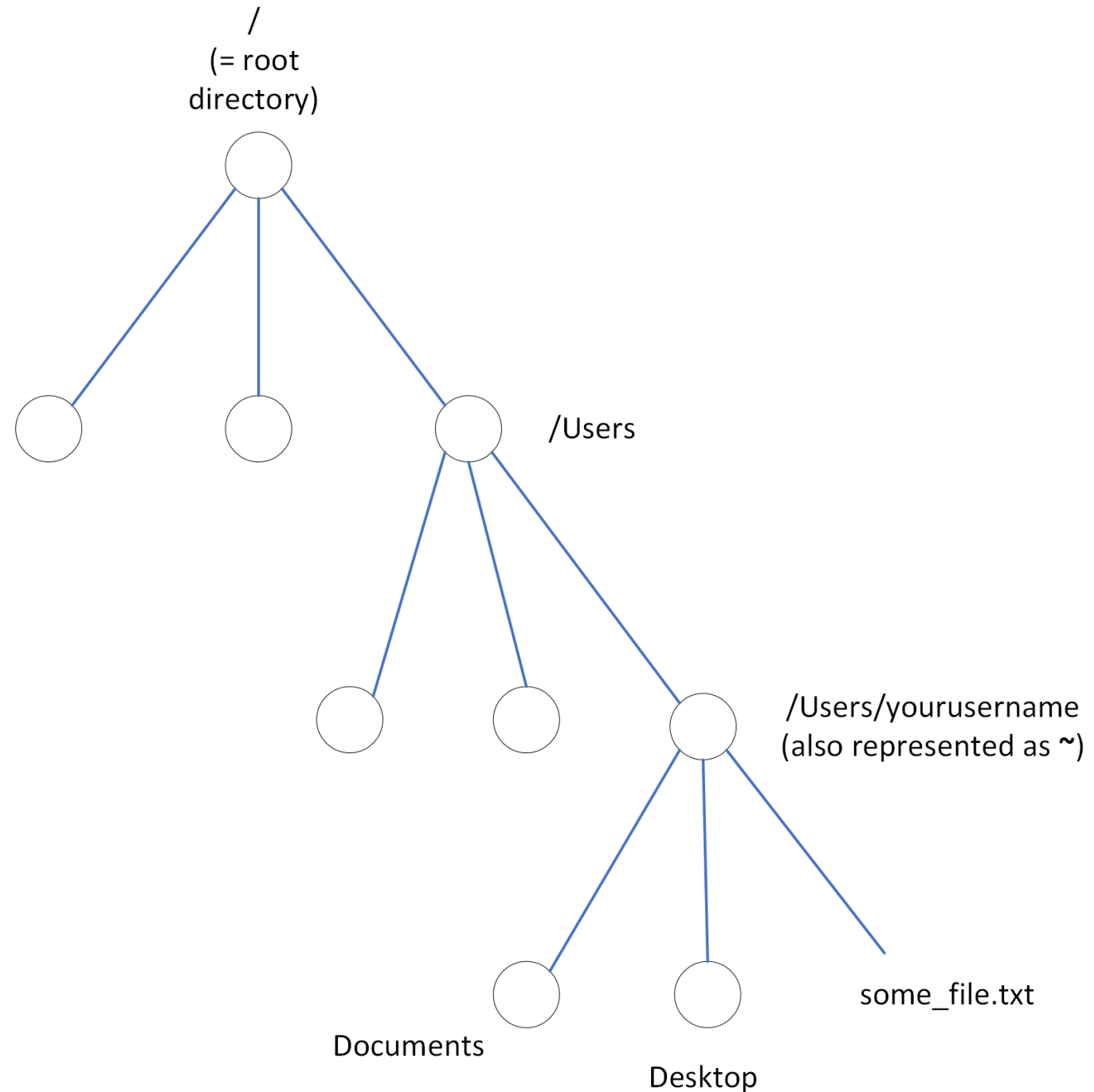
- Cloud services integrated into the operating system have made this a bit more difficult sometimes
- You might not see your Desktop and Documents directories, because they are somewhere else (e.g. OneDrive)
- Even if you see them, they might just be “shortcuts” (as happens in macOS with iCloud Drive)
- Who finds their Desktop and Documents directories in their home directory?

Moving between directories

- You can access these same directories in the GUI
 - macOS: in the Finder, Go > Go to Folder...
 - Windows: Windows key + R > C:\Users\mcarloni (you need to rewrite the path a bit)
- How to change directory? `cd Documents`
- You can inspect the content: `ls`
- How to go back to the parent directory? `cd ..`
- Try it again with another folder

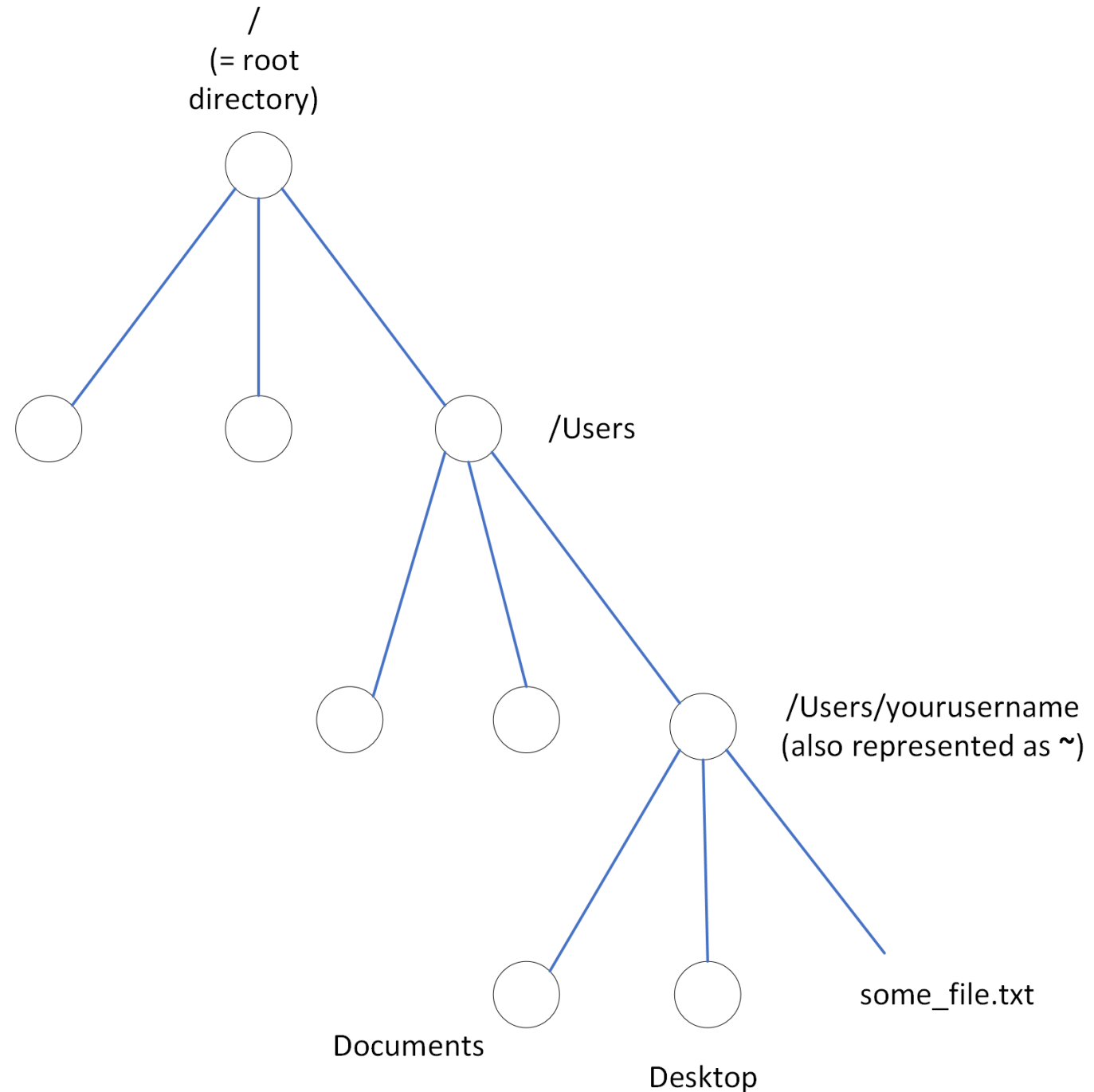
Some „special“ directories

- Let's try to get to the uppermost level of the file system



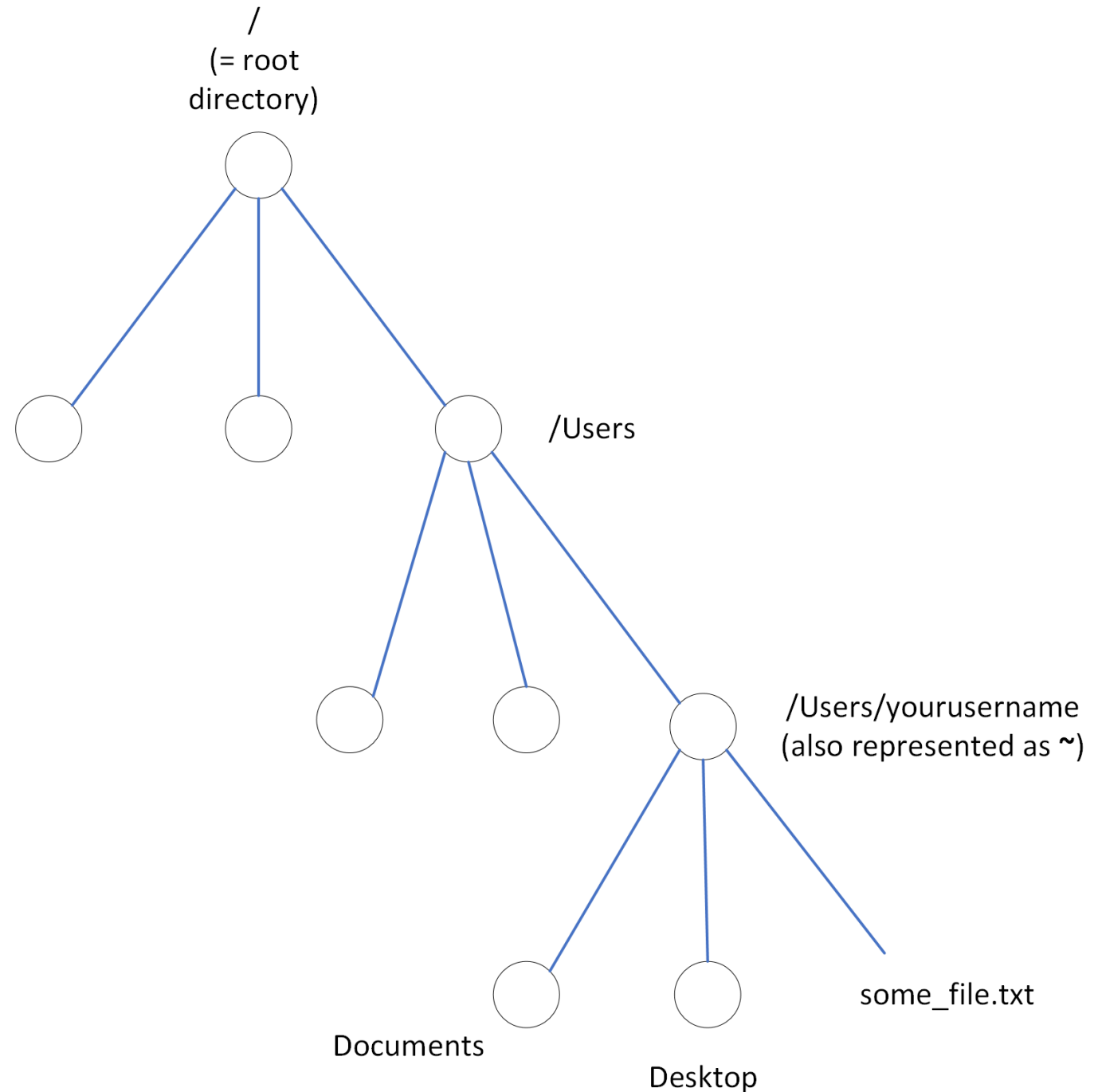
Some „special“ directories

- Let's try to get to the uppermost level of the file system
- `cd /` `cd /c` (for Win)



Some „special“ directories

- Let's try to get to the uppermost level of the file system
- `cd /` `cd /c` (for Win)
- And how do we get back to our home directory?

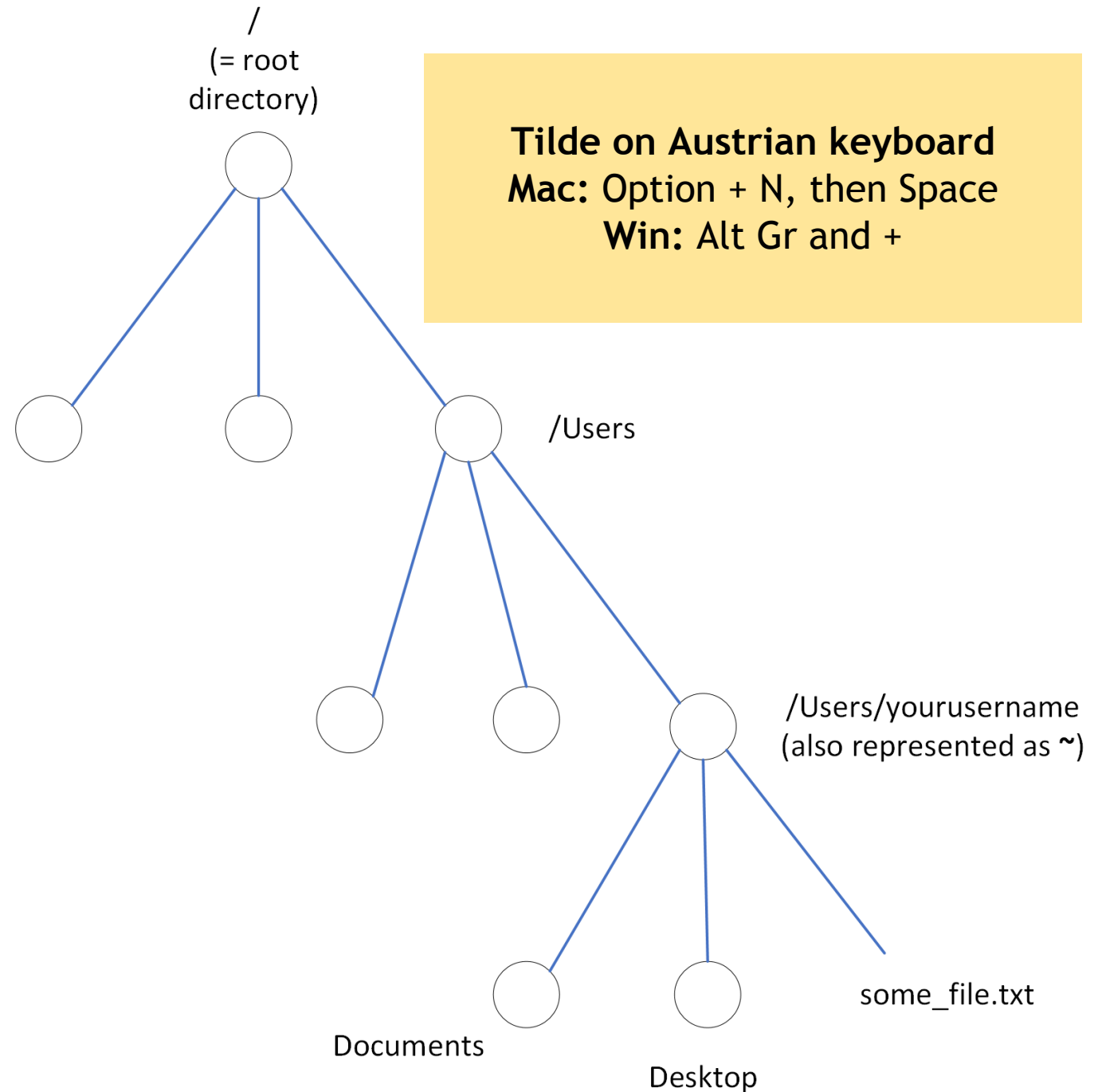


Some „special“ directories

- Let's try to get to the uppermost level of the file system
- `cd /` `cd /c` (for Win)
- And how do we get back to our home directory?
- Two options

`cd`

`cd ~`



Options

- You can modify the behavior of command by including *options/flags/switches*
- Let's try it with the `ls` command
- How do we distinguish between files and directories? `ls -F`
 - Directories will now have `/` at the end
- List content in reverse alphabetical order: `ls -r`
- You can also write options in a long form: `ls --reverse`
 - But this does not always work! `ls --reverse` not working with macOS.

Help/1

- Want to know more about available options?
 - `ls --help`
 - `whatis ls` (short description)
 - `man ls`
 - `help ls`
- Not all options supported by every system!
 - macOS does not support `ls --help` → best choice is `man ls`
 - Git bash on Windows does not support `man ls` → `ls --help`
- How to navigate through man pages?

Navigate in man pages

Key	Action
space	forward one window
b	backward one window
d	forward one half-window
u	backward one half-window
g	go to first line
G	go to last line
/pattern	search forward for pattern
?pattern	search backward for pattern
n/N	next/previous search result
q	exit

Help/2

- Command line reference:
<https://ss64.com/bash/>
- Or just google the command
(followed by unix,
terminal, or the like)

SS64 Linux > How-to > Search

An A-Z Index of the **Linux** command line: bash + utilities.

A

&	Start a new process in the background
alias	Create an alias •
apropos	Search Help manual pages (man -k)
apt	Search for and install software packages (Debian/Ubuntu)
apt-get	Search for and install software packages (Debian/Ubuntu)
aptitude	Search for and install software packages (Debian/Ubuntu)
aspell	Spell Checker
at	Schedule a command to run once at a particular time
awk	Find and Replace text, database sort/validate/index

B

basename	Strip directory and suffix from filenames
base32	Base32 encode/decode data and print to standard output
base64	Base64 encode/decode data and print to standard output
bash	GNU Bourne-Again SHell
bc	Arbitrary precision calculator language
bg	Send to background
bind	Set or display readline key and function bindings •
break	Exit from a loop •
builtin	Run a shell builtin
bzip2	Compress or decompress named file(s)

C

Arguments

- Commands often support arguments
- Arguments tell the command to operate on a specific item
- Other than flags, arguments are not predefined, because they can be files on your drive, phrases etc.
- For example, how can we look into the content of a directory without moving to that directory?
- While you are in your home directory: `ls Documents`

Basic structure of a command

command flag(s) argument(s)

e.g.

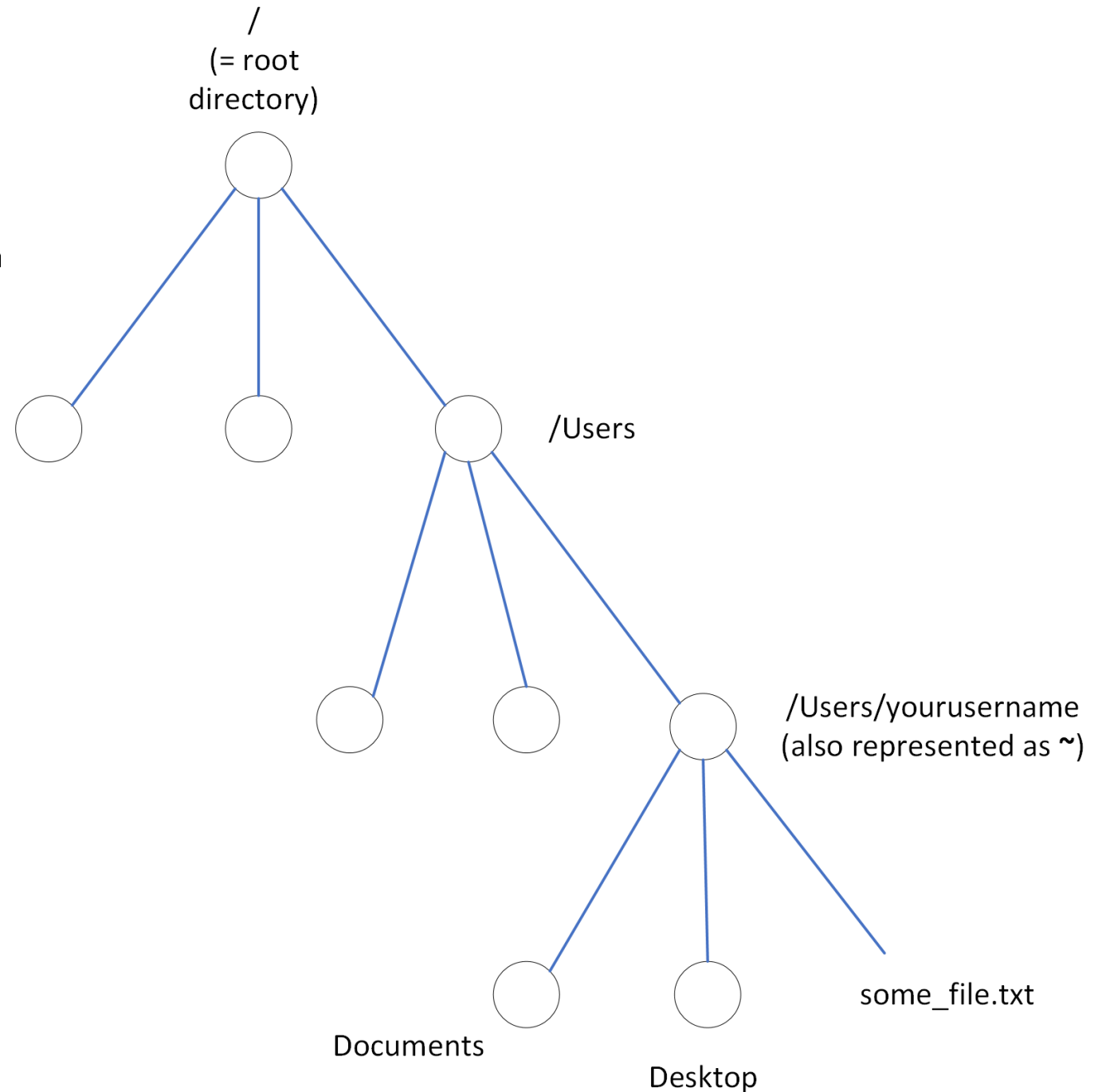
ls -F Documents

Some more notes on options

- Options are case-sensitive
 - `ls -a` $\neq$ `ls -A`
- You can combine options together
 - `ls -l -h`
 - `ls -lh`
- Some options require an argument immediately after them
 - We will see some later ($\rightarrow$ head)

Absolute vs relative paths

- Absolute path (→ pwd)
 - `/Users/mcarloni/some_file.txt`
- Relative path
 - **Documents** (starting from `/Users/mcarloni`)
 - **mcarloni/Documents** (starting from `/Users/`)
 - `.` (if you are in `/Users/mcarloni/Documents`)



Current / parent directory

- The single dot (.) indicates the current directory
 - Running `ls .` is the same as running `ls`
- The parent directory can be indicated by `..`
 - You do not need to know the name of the parent directory, because `..` will always indicate the parent directory relatively to the current working directory (the one you get with `pwd`)
- As we saw before, to go back to the parent directory, you can use `cd ..`
- You can even use `..` in paths
 - `cd ../Documents` (imagining you are in `/Users/mcarloni/Downloads`)
 - `cd ../../..` goes up two directories

Typing commands faster

Auto-complete

- You do not need to type the names of directories and files
- You can just auto-complete by pressing **Tab**
- If more options apply, the system will show you them

Command history

- If you want to re-run a previous command, just press the **arrow up** key (↑) on your keyboard
- You can move up and down your command history with **arrow up** (↑) and **down** (↓)

Cleaning up

- Some useful shortcuts:
 - Delete the whole line: Ctrl+U
 - Move to beginning of line: Ctrl+A
 - Move to end of line: Ctrl+E
- Option + click (Mac only)
- If the screen has become a bit cluttered: clear (or Ctrl+L)
- Press Ctrl+C to abort a command

Working with directories

- Let's go back to the home directory + keep the GUI open
- mkdir test = create directory „test“
 - Avoid spaces in directory names
 - Avoid hyphens at the beginning of directory names
- mkdir -p test1/test2 = create nested directory „test/dir2“
- rmdir test = removes empty directory
- rmdir test1 → does not work! Directory not empty
- rm -rf test1 = removes directory and all its content (caution!)

Work with files

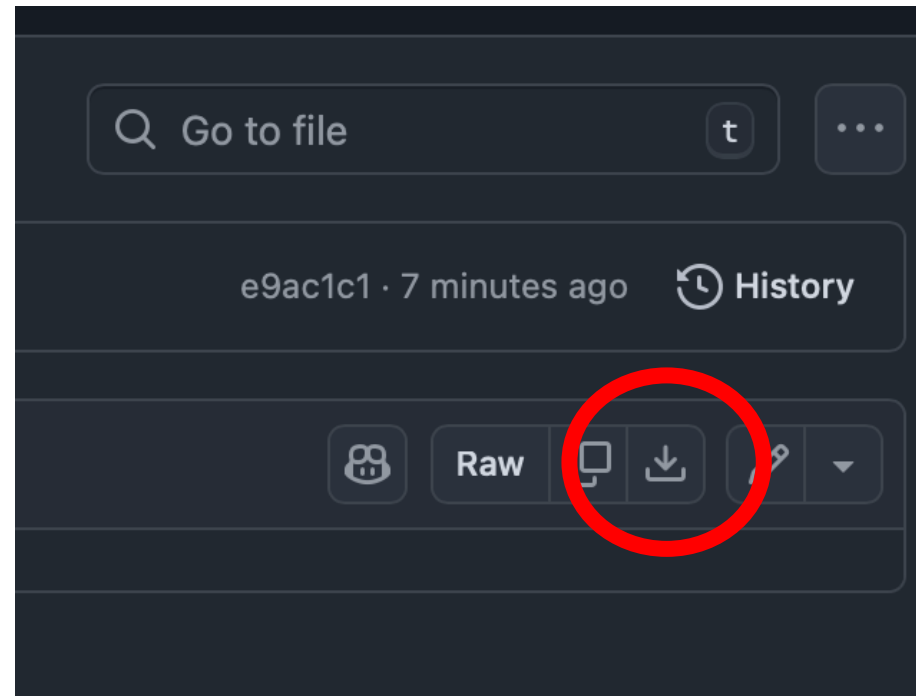
- Easy way to create a new file: `touch dog.txt`
- Copy the file in the same directory:
`cp dog.txt cat.txt`
- Create directory `animals`
- Move a file: `mv dog.txt animals`
- Move into `animals`
- Rename a file: `mv dog.txt fish.txt`
- Move `cat.txt` into `animals`
- Remove a file: `rm fish.txt`
- Remove a file (asking for permission): `rm -i cat.txt`



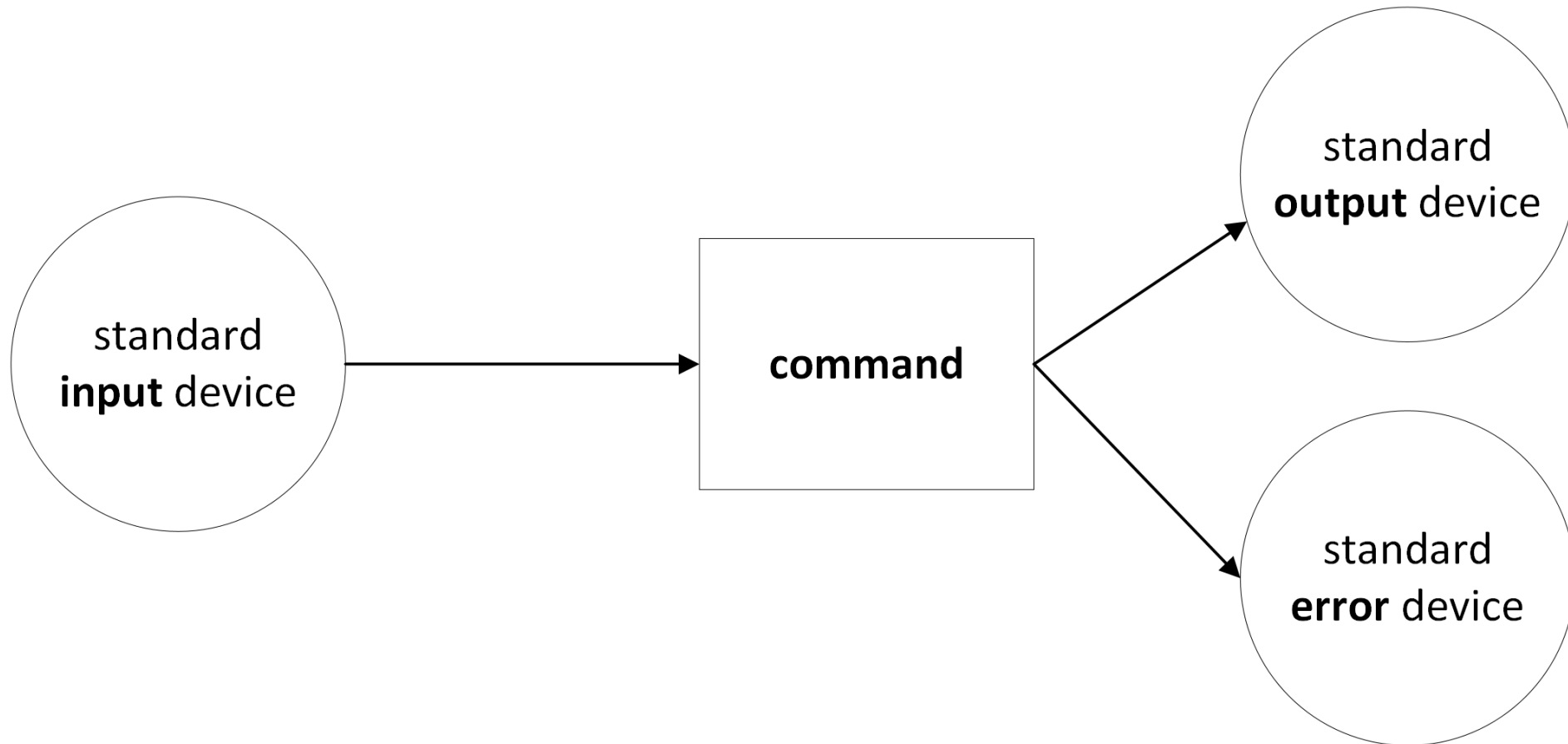
PAUSE

Download

<https://tinyurl.com/dh-vienna-2025w>



Redirection



Input and output

- We usually input data into a command by typing them as arguments with a *keyboard*
- We usually get the output on the *screen/shell interface*
- Could we change this?
- For example, **output to file**: `ls > dir_list.txt`
- This allows us to print out a list of the contents of a directory to a file! 

Append output to a file

- You can also use `>>` to append output to a file, without overwriting that file
- You can try to create a scrapbook and add lines to it as new ideas come in
 - Create a new file named `notes.txt`
 - We will use the **echo** command: echo just outputs on screen the string (sequence of characters) you give (echo "Hello world")
 - `echo "I would like a new car" > notes.txt`
 - We append a second line: `echo "I should definitely buy a new car" >> notes.txt`

How do we look into a file?

- Easiest way: `cat notes.txt`
- More navigable way: `more waste_land.txt`
- Even more functionality: `less waste_land.txt`
 - Less is more 😊
- Get first 10 lines: `head waste_land.txt`
- Get last 10 lines: `tail waste_land.txt`
- Get specific numbers of first/last lines
 - `head -n 20 waste_land.txt`
 - `tail -n 20 waste_land.txt`

Batch processing

- The shell allows for the use of wildcards, such as *
- They work differently from regular expressions
- This different style for finding patterns is called *globbing*
- Let's try it out
 - Check if you are still in the animals directory (pwd)
 - Create some text files: touch dog.txt cat.txt
 - Create some files with other extensions: touch fish.xml capybara.csv
 - Create a directory texts
 - Move all .txt files into the texts directory: mv *.txt texts

Exercise

Imagine you are in a directory that has five files:

dog.txt cat.txt fish.txt
copybara.xml deer.csv

Write a single line in the shell to create a list of all .txt files present in this directory and output it to a file.

Pipes

- You can concatenate different commands by using pipes
- Through a pipe, the output of the first command passes as input of the second command
- The concatenation is called a *pipeline*
- How you write a pipe |
 - Mac Austrian keyboard: Option + 7
 - Windows Austrian keyboard: Alt Gr + 7



Trying out pipes

What is the longest book of the *Iliad* translated by Pope?

- Get length of book 1: `wc book_01.txt`
- Get only number of lines: `wc -l book_01.txt`
- Get number of lines for all texts: `wc -l *.txt`
- Sort them: `wc -l *.txt | sort`
- Or better: `wc -l *.txt | sort -n` (to avoid 10 being sorted before 2)

Shell scripts

- Try out the instructions here: https://github.com/acdh-oeaw/Teaching_CBS4DH/blob/2025W/lectures/command_2.md#scripts

Exercise

Given the test directory `iliad_pope`, create a pipeline that outputs the total number of lines in all `*.txt` files. Use the commands explained in this class. The output should be a single line that looks like this:

21112 total

Regular expressions

- You can use regular expressions in the shell too!
- Suppose you want to find all instances of Trojan / Trojans in book 1 of the Iliad
- The regex pattern would be `Trojans*`
- You can write: `grep "Trojans*" book_01.txt`
- If you add the option `-n`, you also get the line numbers of the occurrences: `grep -n "Trojans*" book_01.txt`
- You can also apply regex to the outputs on the shell using pipes (e.g. find all filenames with a specific pattern):
`ls | grep "book_2\d"`

Extended regex

- To use certain symbols, you need to activate the „extended regular expressions“
 - more specifically, if you want to use ? (zero or one occurrence), + (one or more occurrences) and | (alternative)
- You can do this by adding the option -E
- Or using the command egrep
- For example, to look for „Greeks“ or “Trojans“:
 - `grep -nE "(Greeks|Trojans)" book_01.txt`
 - `egrep -n "(Greeks|Trojans)" book_01.txt`
- And to search in all files you can use the wildcard when specifying the file names: `grep -nE "(Greeks|Trojans)" *.txt`